

期末專題主題報告

基本資訊

項目	內容
專題名稱	隨便吃什麼都好
組別 / 組號	第十組/真不錯
組員姓名	王瑩捷、莊璇容、林姮羽、許卉欣、魏妤安、林少庠

一、專題概述(Planning)

1.1 動機與背景

在現代繁忙的生活節奏下，「選擇困難症」已成為大眾的普遍現象，尤其在面對多樣化的餐飲選擇時，每天決定午餐或晚餐往往耗費大量的時間與心力。這個問題在群體行動（如辦公室訂餐、朋友聚會）中尤為明顯，意見分歧常導致決策效率低下。

此問題在生活中是真實且高頻存在的。我們希望透過開發一個隨機推薦平台，將「決策成本」降至最低。透過數位化的隨機篩選機制，不僅能為使用者提供趣味性的互動體驗，也能打破人們總是吃固定幾間店的「味覺慣性」，探索更多潛在的餐食選擇，將瑣碎的煩惱轉化為輕鬆的遊戲化體驗。

1.2 目標使用者

上班族與大學生：每天生活節奏固定，面臨選擇有限卻又容易感到厭倦的族群。

群體決策者：如部門小組長或聚會發起人，需要一個公正工具來快速達成用餐共識。

喜愛探索者：願意嘗試新口味，但缺乏搜尋動力或特定目標的消費者。

生活極簡主義者：希望減少日常瑣事干擾，將精力集中在更重要決策上的人。

1.3 專題目標

- 消除決策焦慮：透過演算法隨機產生建議，取代長時間的思考與糾結，達成「一鍵決定」。
- 優化篩選機制：提供預算、距離、料理類型等自定義標籤，讓隨機推薦在具備驚喜感的同時，仍符合當下的基本需求。
- 增加探索樂趣：整合地圖與評價功能，鼓勵使用者踏出舒適圈，發掘隱藏在附近的優質店家。

- 提升群體效率: 建立簡單的投票或共識機制, 縮短多人討論餐點的時間, 減少社交壓力。

二、需求分析(Requirements Analysis)

2.1 功能性需求(Functional Requirements)

功能編號	功能名稱	功能描述
F-01	附近餐廳推薦	系統可取得使用者目前位置, 並推薦附近餐廳
F-02	隨機餐廳抽選	使用者可一鍵隨機抽取附近餐廳, 解決選擇困難
F-03	條件篩選	使用者可依距離、價格、料理類型等條件篩選餐廳
F-04	餐廳資訊顯示	顯示餐廳名稱、地址、評分、營業時間等資訊
F-05	收藏與歷史紀錄	使用者可收藏餐廳並查看過去推薦紀錄
F-06	「多人投票決策」模式	使用者可建立一個臨時房間或產生分享連結, 讓多人針對隨機抽出的幾家候選餐廳進行快速投票, 以多數決定最終去處。

2.2 非功能性需求(Non-functional Requirements)

類別	需求描述
效能	系統需在 3 秒內完成附近餐廳搜尋與推薦結果顯示
安全性	使用者定位資料需經授權後使用, 並避免長期儲存精確位置資訊
易用性	提供「一鍵推薦」功能, 讓使用者能在 10 秒內完成選擇
相容性	系統需支援手機與桌面瀏覽器(RWD 設計)

2.3 範圍限制 (Out of Scope)

- 本次不包含: 本次不包含: 即時導航功能 (僅提供地圖連結)
 - 本次不包含: 外送、訂位或付款功能
 - 本次不包含: AI 個人化深度學習推薦 (僅使用基本條件與隨機邏輯)
-

三、技術規劃 (Technology Planning)

3.1 技術選型

層面	技術選擇	說明
前端	HTML / CSS / JavaScript (純前端, 無框架)	課堂統一規範
後端	Python + Flask	課堂統一規範
資料庫	SQLite	輕量級關聯式資料庫, 無需額外安裝伺服器, 非常適合小型專題與 Flask 搭配。
地圖/餐廳 API	Google Places API	取得附近餐廳資訊、評分及經緯度的業界標準工具 (有免費額度)。
前端樣式	Bootstrap 5	快速達成 RWD 響應式設計, 讓系統在手機與電腦上都能流暢使用。

3.2 開發工具與協作方式

- 版本控制: GitHub, 說明: 核心代碼代管與版本紀錄, 避免開發 Flask 時程式碼寫爛回不去。
 - 溝通管道: LINE, 說明: 主要的即時通訊工具。利用「公告」功能置頂進度, 並建立「相簿」儲存 UI 設計草圖。
 - 進度看板: Notion / Trello, 說明: 搭配 LINE 使用, 將複雜的需求清單 (F-01~F-05) 視覺化。
 - 編輯器: VS Code, 說明: 必備的 IDE, 建議安裝 Prettier 插件確保小組成員的代碼格式一致。
-

四、專題時程規劃(Project Schedule)

階段一：規劃 Planning × Requirements × Design

日期	預計完成工作	產出物	負責
04/30-05/04	確定主題與需求：定義隨機邏輯(地區、預算、類型)	主題報告投影片	全組
05/05-05/08	系統架構與介面設計：規劃資料庫結構與UI/UX 流程 架構圖、Mockup 預覽圖	架構圖、Mockup 預覽圖	全組

投影片需包含：① 專題介紹 ② 功能規劃 ③ 預估畫面(自己設計或 AI 生成圖)

階段二：實作 Implementation × Testing

日期	預計完成工作	負責組員
05/09-05/18	前端開發與資料串接：建立轉盤或隨機按鈕、串接 Google Maps 或自建資料庫	全組
05/19-05/26	後端與功能優化：會員收藏、評論紀錄或特定條件篩選	全組
05/27-06/02	使用者測試與除錯：進行 Bug 修正與介面微調(美化 CSS)	全組

階段三：期末展示 Demo

日期	預計完成工作	負責
06/03-06/07	報告投影片製作：包含市場分析、開發亮點、Demo 影片	全組
06/08-06/10	報告演練與 Demo 確認：確保現場演示時不會發生意外	全組
06/11	最終平台發表與報告	全組

五、風險評估(Risk Assessment)

風險	發生可能性	影響程度	因應策略
Google Places API 額度超標或產生預期外費用(開發或展示期間頻繁呼叫 API)	中	高	於 Google Cloud 控制台設定預算上限與用量警報。開發測試初期盡量使用假資料(Mock Data) 測試前端, 減少真實 API 呼叫; 後端可將查詢過的相同條件結果暫存入 SQLite 降低重複請求。
使用者拒絕提供定位權限或定位失準(導致無法取得「附近餐廳」)	高	高	在系統請求定位權限時, 加入友善的文字說明其必要性。同時提供「手動輸入位置(例如直接輸入: 逢甲大學)」的備用方案, 確保無定位時系統仍能正常運作。
篩選條件過於嚴苛導致無餐廳可推薦(如設定半夜時段且距離極短, 導致抽籤失敗)	中	中	在介面上設定防呆機制或防護網(Fallback)。若無符合條件的結果, 系統可給予友善提示(如: 「附近沒有符合的餐廳喔, 要不要放寬條件試試?」), 或自動微幅擴大搜尋半徑。
技術卡關導致期末時程延宕(例如 API 串接不順利或前後端整合失敗)		高	嚴格執行 MVP(最小可行性產品)策略。優先確保「獲取定位 → 隨機抽出一間餐廳」的最核心功能可運行; 「會員收藏紀錄」、「進階條件篩選」等功能列為加分項, 視進度彈性調整。

六、組員分工

規劃階段(全組共同完成)

規劃階段由全組一起討論，共同完成以下項目：

- ☐ 專題主題與目標確認
- ☐ 功能清單(需求分析)
- ☐ 系統架構與技術選型
- ☐ 畫面設計與 Wireframe / AI 預估畫面
- ☐ 週次時程與任務分配

實作階段(各組員負責獨立功能)

每位組員至少負責一個完整功能模組(前端頁面 + 後端 Flask 路由)。

姓名	負責功能模組	對應功能編號	說明
王瑩捷	收藏與歷史紀錄	F-05	使用者可收藏餐廳並查看過去推薦紀錄
莊琬容	條件篩選	F-03	使用者可依距離、價格、料理類型等條件篩選餐廳
林姮羽	餐廳資訊顯示	F-04	顯示餐廳名稱、地址、評分、營業時間等資訊
許卉欣	附近餐廳推薦	F-01	系統可取得使用者目前位置，並推薦附近餐廳
魏妤安	隨機餐廳抽選	F-02	使用者可一鍵隨機抽取附近餐廳，解決選擇困難
林少庠	「多人投票決策」模式	F-06	使用者可建立一個臨時房間或產生分享連結，讓多人針對隨機抽出的幾家候選餐廳進行快速投票，以多數決定最終去處。

功能編號請對應「二、需求分析」的功能清單，每項功能需有明確負責人且不重疊。

本報告依照課堂 *SDLC* 開發流程範本製作。